

DevLabs Builder OS & Founder OS

Talent profiles, storage, matching, and discovery — implementation reference for LLM handoff

Scope: Builder OS + Founder OS only (excludes admin dashboard / club member vector search)

Generated: June 4, 2026

Executive summary: Builder profiles are **structured MongoDB documents** (not vector embeddings) joined with `ProjectRecord` proof-of-work. Founder talent discovery uses a **deterministic weighted scoring engine** in `builderSearch.ts`. The AI agents (`builder_chat`, `founder_chat`) **orchestrate** profile updates and search triggers; they do **not** embed profiles or perform semantic retrieval for matching. Optional **Supermemory** stores conversational memory per user; optional **OpenRouter LLM** evaluates profile quality and parses resumes — still written back to Mongo as fields, not embeddings.

Table of contents

1. Data model: what is a “builder profile”
2. How profiles are created and updated (Builder OS)
3. Embeddings: what exists vs what talent OS uses
4. Profile scoring and quality (deterministic + LLM)
5. Founder role briefs and search prerequisites
6. Matching algorithm (`rankBuildersForOpportunity`)
7. Search execution and persistence (`Shortlist`, `MatchRecord`)
8. Founder discovery UX and AI agent integration
9. End-to-end flows
10. Key files index

1. Data model: builder profile

Primary collection: `BuilderProfile`

File: `src/models/talent/BuilderProfile.ts`

One document per builder, keyed by `userId` (WorkOS/User) and/or `email`. Core fields:

- **Identity:** name, email, headline, bio, location, universityOrCompany
- **Hiring signals:** `rolePreference[]` (skills/roles), `preferredWorkType[]`, availability (`availableNow`, `hoursPerWeek`, `remotePreference`)
- **Links:** github, linkedin, portfolio, resume, devpost

- **Computed caches:** profileCompletion (profileScore, proofScore, matchScore, missingItems, eligibility), profileQuality (LLM or deterministic evaluation blob)
- **Visibility:** verificationStatus, visibilityStatus (public | matched_only | hidden)
- **legacyRefs:** pointers to legacy User/Application docs from bootstrap

Proof-of-work: ProjectRecord

File: `src/models/talent/ProjectRecord.ts`

Many projects per builder (`builderId`). Matching and founder review read project text heavily:

- projectName, description, problemSolved, builderContribution, techStack[], contributionTags[]
- links (github, devpost, demo, screenshots), source, sourceId
- verificationStatus: imported_unverified → builder_confirmed → peer_confirmed / admin_verified / founder_verified

Founder hiring context: Opportunity

File: `src/models/talent/Opportunity.ts`

Role brief / search definition: company, roleTitle, builderWillDo, skillsNeeded[], hireType/workType, timeline, budget, locationPreference, status (draft → shortlisted → ...).

Search artifacts

Model	Purpose
Shortlist	Per-opportunity ranked preview: candidates[] with anonymous labels, match scores, unlocked flag
MatchRecord	Per (builderId, opportunityId) pipeline row: scores, reasoning, status (generated → intro_requested → trial → hired)

2. Builder profile lifecycle (Builder OS)

Creation

- **On first authenticated API call:** `resolveAuthedBuilder` in actionsHandler creates a minimal BuilderProfile if none exists (email from JWT, default availability).
- **Bootstrap script (ops):** `POST /api/talent/bootstrap-builders` copies User + Application into BuilderProfile with legacyRefs; uses `computeProfileCompletion` only.

Update paths (no embedding step)

Path	What gets written
------	-------------------

Builder Agent builder_chat tools	update_profile_basics, update_links, update_availability, import_project, evaluate_profile
Direct actions	update_profile_basics, update_links, update_availability, update_work_preferences, claim_profile
Resume upload	PDF → Cloudinary URL on links.resume; LLM parse → headline, bio, rolePreference, ProjectRecords
Project import	GitHub API or Devpost markdown + LLM extract → ProjectRecord upsert by sourceId URL
Dashboard load	get_builder_dashboard recomputes profileCompletion; may run deterministic profileQuality

Builder OS UI consumption

- BuilderOSDashboard.tsx loads via get_builder_dashboard
- Agent tab: builder_chat with chat history; refreshes builder object from response
- Home/Profile tabs show profileCompletion, profileQuality.suggestedFixes, projects, intros, trials, calls

After most writes, updateBuilderScores / computeBuilderScores recomputes profileCompletion and optionally LLM profileQuality.

3. Embeddings — critical distinction

Talent OS (Builder + Founder): NO vector embeddings for profiles

There is **no** call to getVectorStore() , MongoDB Atlas Vector Search, or Cloudflare embeddings anywhere under src/lib/talent/. Founder search does not embed the opportunity or builder corpus.

Matching is **keyword/fuzzy token overlap + weighted heuristics** over in-memory builder + project documents loaded from Mongo.

What “embedding” means elsewhere in the repo (out of scope)

Separate admin/club pipeline uses src/lib/vectorStore.ts (Cloudflare BGE + MongoDB Atlas) for Application/resume chunks via /api/search/vector-only and RAG. That path is **not** used by Builder OS or Founder OS talent discovery.

Supermemory (optional, not profile embedding)

File: src/lib/talent/supermemory.ts

- Container tags: builder-{builderId} , founder-{founderId}

- `getBuilderMemoryProfile` / `getFounderMemoryProfile`: profile API returns static + dynamic memory strings
- Injected into agent **system prompts** (up to 5 static + 3 dynamic bullets)
- `addBuilderMemory` / `addFounderMemory`: fire-and-forget text memories after tool actions (e.g. "updated availability")
- Requires `SUPERMEMORY_API_KEY`; if missing, agents work without cross-session memory

Supermemory is **conversational memory**, not a searchable embedding index of builder profiles for matching.

4. Profile scoring and quality

profileCompletion — deterministic (matching.ts)

`computeBuilderScores(builder, projects):`

- **profileScore (0–100)**: checklist — name, email, availability fields, rolePreference, preferredWorkType
- **proofScore (0–100)**: project count, links, builderContribution length, builder resume/github/portfolio links
- **matchScore / eligibility**: combined readiness; priority if matchScore ≥ 80
- **missingItems**: human-readable gaps for Builder Agent suggestions

profileQuality — deterministic fallback + optional LLM

File: `src/lib/talent/profileQuality.ts`

- `evaluateDeterministicQuality`: rule-based score from headline/bio length, projects, contributions, links
- `evaluateBuilderProfileQuality`: if OpenRouter configured, sends trimmed profile+projects JSON to LLM; returns structured issues, suggestedFixes, founderClarity score; else deterministic
- Stored on `builder.profileQuality` with source `llm` or `deterministic`
- Used in matching as `scoreProfileQuality` (overallScore + founderClarity blended)

Resume processing (structured extract, not vectors)

`src/lib/talent/resumeParser.ts`: pdf-parse → OpenRouter JSON extract → updates BuilderProfile fields + creates ProjectRecords. No vector store write.

5. Founder role brief & search gates

Role brief storage

Opportunity document, created/updated by:

- Founder Agent tools: create_role_brief, update_role_brief (sanitized via founderRoleBrief.ts — no placeholder company, merge hints from user text)
- Legacy actions: create_role_brief, update_role_brief, enhance_role_brief_field

When search is allowed

canRunPreviewAnyway(opportunity) in founderSearchQuality.ts requires:

1. roleTitle (non-placeholder)
2. builderWillDo OR startupSummary
3. skillsNeeded.length > 0

Full REQUIRED_FIELD_CHECKS also include workType, timeline, location — used for UI completeness, not the minimal search gate.

6. Matching algorithm

File: src/lib/talent/builderSearch.ts — function rankBuildersForOpportunity(builders, projectsByBuilder, opportunity, limit=12)

Candidate pool

```
BuilderProfile.find({
  verificationStatus: { $ne: 'rejected' },
  visibilityStatus: { $ne: 'hidden' }
}).limit(2000)
```

All projects for those builders loaded in one query, grouped by builderId in a Map.

Per-builder component scores (0–1 each)

Component	Weight in matchScore	Logic (summary)
skillFit	30%	Tokenize role skills + title; fuzzy match against headline, bio, rolePreference, project text/stacks; SKILL_ALIASES (e.g. golang/go)
proofRelevance	25%	Verified projects preferred; count, contribution clarity, tech stack presence; fallback to proofScore
availabilityFit	15%	availableNow, hoursPerWeek, remote vs opportunity location
workTypeFit	10%	preferredWorkType vs opportunity.workType/hireType aliases

domainRelevance	10%	Keyword overlap between opportunity summary/industry/builderWillDo and project text
profileQuality	10%	profileQuality.overallScore + founderClarity, normalized 0–1

Composite scores

- **matchScore** = round(weighted sum × 100)
- **profileStrength** = separate heuristic (quality, proof, links, project count) via profileStrengthScore()
- **rankingStrength** = matchScore × 0.7 + profileStrength × 0.3
- **matchLabel**: ≥80 Strong Match, ≥65 Good Match, else Possible Match

Hard filter

If opportunity has specific requirements (skills, roleType, or roleTitle) AND skillFit < 0.25 AND domainRelevance < 0.2 → builder **excluded** from results.

Sort & limit

Sort by rankingStrength, then profileStrength, matchScore, skillFit, proofRelevance. Return top 12.

Derived copy (not LLM)

- whyTheyMatch — template sentences from component thresholds
- proofSummary — anonymous project/stack summary
- availabilitySummary — from availability fields
- signals — low/medium/high buckets per dimension

7. Search execution & persistence

Two entry points, same core logic

1. `action: run_builder_search` or `rerun_search` — explicit API from Founder OS UI
2. Founder Agent tool `run_search` — identical ranking inside founder_chat tool executor

After ranking

1. `toAnonymousCandidates(ranked, N)` — anonymousLabel like "Full-stack · Strong Match" (role type + label; names hidden in preview metadata)
2. For each result: `MatchRecord.findOneAndUpdate` upsert with matchScore, profileStrength, matchLabel, anonymousLabel, signalScores, reasoning (whyTheyMatch), status generated unless already in pipeline
3. `Shortlist.findOneAndUpdate`: totalMatches, strongMatchCount, candidates[], previewGeneratedAt, unlocked=true (current code auto-unlocks)

4. Opportunity.status → shortlisted
5. `buildFullCandidatesForShortlist` — hydrates real names, projects, verification labels for founder kanban/drawer (founderCandidate.ts)

Rerun behavior

On `rerun_search`, pipeline builders (status not generated/closed/rejected) may be merged back into shortlist even if they fell out of top ranks.

8. Founder OS discovery & AI agent

Founder Agent does NOT semantically search builders

The founder_chat LLM:

- Collects/refines role brief via tools (`get_role_brief`, `create/update_role_brief`)
- Triggers `run_search` tool → calls same `rankBuildersForOpportunity` as `run_builder_search`
- Reads results via `get_shortlist`, `get_candidate_details`
- Can request_intro (writes `IntroRequest`, notifies builder)

The agent's "Scanning proof-of-work talent graph" UI copy is branding; execution is deterministic Mongo load + heuristic rank.

UI flow (FounderRoleIntakeChat)

1. Mount → `sendToAgent('__init__')` → founder_chat with synthetic "just signed up" or "existing role" message
2. Agent may call `get_all_roles`, `create/update brief` tools
3. If agent message suggests running search AND `canRunPreviewAnyway` → `triggerSearch()` calls `run_builder_search` (separate HTTP call, not only tool)
4. On success → `onSearchCompleted(opportunityId)` → Founder kanban / workspace

FounderOSDashboard

- `get_founder_dashboard` — searches, shortlists, pipeline
- `rerun_search` — refresh matches for active opportunity
- `FounderCandidateDrawer` — full candidate from `buildFullCandidateCard`
- Non-agent actions: `request_intro`, `unlock_shortlist` (legacy; search now auto-unlocks), `generate_trial_project` (LLM for trial copy only)

Anonymous vs unlocked shortlist

Shortlist stores anonymous labels in `candidates[]`. When unlocked, `get_shortlist` / `buildFullCandidatesForShortlist` expose builder names, projects, proof labels. Founder Agent `get_candidate_details` requires unlocked shortlist.

9. Builder Agent vs matching

Builder Agent improves **inputs** to matching (profile fields, projects, quality scores) but does not run founder search. `fetch_intros` / `fetch_work_trials` / `fetch_messages` read pipeline state only.

When `evaluate_profile` tool runs, `profileQuality` on `BuilderProfile` updates → affects future founder `matchScore` via `profileQuality` component.

10. End-to-end diagrams

Builder profile → searchable corpus

```
Builder signs in
→ BuilderProfile (Mongo)
→ Projects via agent import / resume / manual
→ computeBuilderScores + optional LLM profileQuality
→ Stored as plain text fields (NO embedding index)
```

Founder search

```
Founder defines Opportunity (chat or editor)
→ canRunPreviewAnyway?
→ Load ≤2000 builders + all their projects
→ rankBuildersForOpportunity (heuristic)
→ MatchRecord + Shortlist persist
→ Founder UI kanban / candidate drawer
```

Where AI is used vs not

Step	AI?	Mechanism
Store builder profile	No	Mongo documents
Embed profile for search	No	Not implemented in talent OS
Rank builders for role	No	<code>builderSearch.ts</code> heuristics
Parse resume / import Devpost	Yes	OpenRouter JSON extract
Profile quality evaluation	Optional	OpenRouter or deterministic
Founder/Builder chat orchestration	Yes	OpenRouter tool-calling agent
Agent memory across sessions	Optional	Supernote text memories
Trial project copy	Optional	<code>founderTrialProject</code> LLM

11. Key files index

File	Role
src/models/talent/BuilderProfile.ts	Builder schema
src/models/talent/ProjectRecord.ts	Proof-of-work projects
src/models/talent/Opportunity.ts	Founder role brief
src/models/talent/Shortlist.ts	Search results snapshot
src/models/talent/MatchRecord.ts	Per-builder pipeline row
src/lib/talent/matching.ts	Profile completion / proof scores
src/lib/talent/builderSearch.ts	Matching/ranking engine
src/lib/talent/profileQuality.ts	LLM + deterministic quality
src/lib/talent/founderCandidate.ts	Full candidate cards for founders
src/lib/talent/founderSearchQuality.ts	Brief completeness, search gates
src/lib/talent/resumeParser.ts	Resume → structured profile
src/lib/talent/supermemory.ts	Agent memory (not match index)
src/lib/agent/actionsHandler.ts	run_builder_search, founder_chat run_search, builder tools
src/components/builder/BuilderOSDashboard.tsx	Builder OS shell
src/components/founder/FounderRoleIntakeChat.tsx	Founder agent + search trigger
src/components/founder/FounderOSDashboard.tsx	Founder pipeline UI

For downstream LLMs: treat talent discovery as a **batch heuristic ranker over Mongo documents**. To add true semantic matching, you would need a new pipeline (embed Opportunity + builder corpus, vector index) — not present in Builder/Founder OS today.